

NATS - Cheat Sheet

Contents

1. [The three layers](#)
2. [NATS Core](#)
3. [JetStream](#)
4. [KV and Object Store](#)
5. [NATS vs Kafka vs RabbitMQ](#)
6. [Clustering and topology](#)
7. [Security](#)
8. [Java client](#)
9. [Best practices](#)
10. [Common gotchas](#)
11. [Quick reference](#)

NATS is a lightweight, high-throughput messaging system. Created by Derek Collison in 2010, written in Go. The core protocol does pub/sub and request/reply over plain TCP. JetStream adds persistent streams on top. The same binary handles both.

Comparable to Kafka and RabbitMQ in scope but built with a different philosophy: small, simple to operate, designed for cloud-native deployments.

1. The three layers

NATS ships as three layers, each building on the one below.

Layer	What it gives you
NATS Core	Pub/sub, request/reply. At-most-once delivery. No persistence. Sub-millisecond latency.
JetStream	Persistent streams, replay, consumer groups, exactly-once delivery. Built into the same binary.
KV + Object Store	Built on JetStream. Key-value lookups and large object storage.

You pick the layer by configuration. Same `nats-server` binary, same protocol.

2. NATS Core

Pub/sub with subject-based routing. Publishers send messages to a subject. Subscribers receive messages on subjects they're subscribed to.

```
publisher -> "orders.created" -> server -> subscribers on "orders.created"
```

Subjects are hierarchical, dot-separated:

```
orders.created
orders.updated
orders.us.created
users.42.profile.updated
```

Wildcards in subscriptions:

- `*` matches one token: `orders.*` matches `orders.created` but not `orders.us.created`.
- `>` matches one or more tokens, must be the last token: `orders.>` matches both.

```
# Publish
nats pub orders.created '{"id":42}'

# Subscribe
nats sub "orders.*"
nats sub "orders.>"
```

Request / reply

Synchronous-style RPC over the messaging fabric. The publisher includes a reply-to subject. The replier sends back on that subject.

```
# Server
nats reply ping "pong"

# Client
nats request ping ""           # returns "pong"
```

In code, the client supplies a timeout. If no reply arrives, the call fails.

Queue groups

Multiple subscribers join a queue group. Each message goes to exactly one of them. Round-robin load balancing across consumers.

```
nats sub "jobs" --queue workers
nats sub "jobs" --queue workers
nats sub "jobs" --queue workers
```

Publishing to `jobs` sends each message to one of the three workers. The standard pattern for stateless worker pools.

Delivery semantics

NATS Core is at-most-once. If a subscriber is down when a message is published, the message is gone. Use JetStream when you need durability.

3. JetStream

The persistence layer. Adds streams, consumers, replay, acknowledgement, retention.

Streams

A stream captures messages on configured subjects and stores them on disk.

```
nats stream add ORDERS \
  --subjects "orders.>" \
  --retention limits \
  --max-msgs 1000000 \
  --max-age 7d \
  --storage file \
  --replicas 3
```

Retention policies:

- `limits`: keep up to `max_msgs`, `max_bytes`, or `max_age`.
- `interest`: keep messages while at least one consumer wants them.
- `workqueue`: delete a message after it's acked. Each message goes to one consumer.

Storage:

- `file`: persisted to disk.
- `memory`: lives in RAM, fast, lost on restart.

Replication (R=3) keeps the stream on 3 nodes with Raft consensus.

Consumers

A consumer is a view of a stream. Two kinds:

- **Push consumer**: NATS sends messages to a subject as they arrive. Closer to NATS Core feel.
- **Pull consumer**: client fetches batches. Better for backpressure and rate control.

```
# Pull consumer
nats consumer add ORDERS workers \
  --pull \
  --ack explicit \
  --max-deliver 3 \
  --filter "orders.created"
```

Acknowledgement modes:

- `none`: no ack needed.
- `all`: acking message N acks everything up to N (good for in-order processing).
- `explicit`: each message must be individually acked.

`--max-deliver 3` retries up to 3 times before giving up on a message.

Replay

Read the stream from any point:

- From now (default).
- From the start.
- From a specific sequence number.
- From a timestamp.

Useful for backfilling new services from existing events.

4. KV and Object Store

Built on JetStream. Same persistence and replication.

Key-Value

```
nats kv add CONFIG
nats kv put CONFIG feature.flag.dark_mode true
nats kv get CONFIG feature.flag.dark_mode
nats kv watch CONFIG "feature.>"
```

KV updates produce events. Subscribers can react to config changes in real time.

Per-key history (last N versions), TTL, atomic compare-and-swap for distributed coordination.

Object Store

Larger objects (files, blobs) chunked and stored as JetStream messages.

```
nats object add ASSETS
nats object put ASSETS profile.png ./profile.png
nats object get ASSETS profile.png > out.png
```

Niche but useful for sharing files across services without standing up S3 for small workloads.

5. NATS vs Kafka vs RabbitMQ

Aspect	NATS Core	NATS JetStream	Kafka	RabbitMQ
Persistence	None	Yes	Yes	Optional
Throughput	Very high	High	Very high	Medium
Latency	Sub-ms	Low	Low to medium	Low
Subject hierarchy	Yes, with wildcards	Yes	No (topics flat)	Yes (exchanges + bindings)
Consumer model	Pub/sub + queue groups	Streams + consumers	Topics + consumer groups	Queues + exchanges
Replay	No	Yes (full or partial)	Yes	No
Cluster topology	Cluster, supercluster, leaf	Same	Brokers + ZK/KRaft	Cluster, federation
Java client	nats.java	nats.java	kafka-clients	amqp-client

NATS Core is the right pick for ephemeral pub/sub or RPC over messaging. JetStream covers most of what Kafka and RabbitMQ do with simpler operations. Kafka wins on raw throughput and ecosystem (Kafka Streams, Connect). RabbitMQ wins on AMQP compatibility and complex routing.

6. Clustering and topology

Four deployment shapes.

Shape	When to use
Single server	Dev, testing, small workloads
Cluster	3+ nodes in one location, full mesh, high availability
Supercluster	Multiple clusters across regions, gossip-routed
Leaf node	Edge or local cluster that bridges to a main cluster

Leaf nodes are powerful for IoT and edge: local clients connect to a leaf, the leaf forwards across the WAN to the main cluster. Local subjects don't leave the leaf unless explicitly exported.

7. Security

Four auth options.

- **None:** open to anyone. Dev only.
- **Tokens or username/password:** simple, static credentials.
- **NKeys:** public-key cryptography. The modern default. Each user has a keypair, the server holds public keys.
- **Decentralized JWT:** accounts and users defined as signed JWTs. Used in the multi-tenant `nsc` /operator model.

NATS has a strong account model. An “account” is a tenant. Subjects, streams, KV buckets are scoped per account. Cross-account communication requires explicit exports and imports.

8. Java client

```
<dependency>
  <groupId>io.nats</groupId>
  <artifactId>jnats</artifactId>
  <version>2.x</version>
</dependency>
```

Core pub/sub

```
try (Connection nc = Nats.connect("nats://localhost:4222")) {
  // Publish
  nc.publish("orders.created", "{\"id\":42}".getBytes());

  // Subscribe (sync)
  Subscription sub = nc.subscribe("orders.*");
  Message msg = sub.nextMessage(Duration.ofSeconds(1));
  System.out.println(new String(msg.getData()));

  // Subscribe (dispatcher = async)
  Dispatcher d = nc.createDispatcher(m ->
    System.out.println("got: " + new String(m.getData())));
  d.subscribe("orders.*");

  // Request / reply
  Message reply = nc.request("ping", null, Duration.ofSeconds(1));
}
```

JetStream

```
Connection nc = Nats.connect("nats://localhost:4222");
JetStream js = nc.jetStream();

// Publish
PublishAck ack = js.publish("orders.created", "{\"id\":42\"}.getBytes());
System.out.println("seq: " + ack.getSeqno());

// Pull subscriber
PullSubscribeOptions opts = PullSubscribeOptions.builder()
    .durable("worker")
    .build();
JetStreamSubscription sub = js.subscribe("orders.created", opts);

List<Message> messages = sub.fetch(10, Duration.ofSeconds(5));
for (Message m : messages) {
    process(m);
    m.ack();
}
```

9. Best practices

- **Pick the right layer.** Use Core for ephemeral pub/sub and RPC. Use JetStream when you need persistence or replay.
- **Use subject hierarchies.** `domain.entity.action[.tag]` reads well and works with wildcards.
- **Set explicit retention policies.** A stream with no limits grows forever.
- **Replicate JetStream streams** when correctness matters. R=3 with Raft.
- **Use queue groups for worker pools.** Save yourself the hand-rolled load balancer.
- **Acknowledge explicitly for JetStream consumers.** Auto-ack hides delivery failures.
- **Set `max_deliver` on consumers.** Stops poison messages from looping forever.
- **Use leaf nodes for edge or per-cluster isolation.** Local subjects stay local.
- **Scope sensitive data with accounts.** One tenant per account.
- **Monitor stream and consumer state.** `nats stream info`, `nats consumer info`, `nats events`.

10. Common gotchas

- **NATS Core is at-most-once.** A subscriber down at publish time misses the message forever. Use JetStream for durability.
- **> wildcard must be last.** `orders.>.us` is invalid. `orders.us.>` is fine.
- **Queue groups don't apply to JetStream pull consumers.** JetStream consumers fan out their own way.
- **Subject explosion.** A subject per user (`user.42.event`) creates millions of subjects. JetStream handles this fine but be deliberate.

- **JetStream replication needs odd numbers.** R=1 (no HA), R=3, R=5 are the practical choices. Raft needs an odd count for quorum.
- **Stream limits silently drop messages.** If `max_msgs` is hit and retention is `limits`, the oldest messages are evicted. Watch the discard policy.
- `max_deliver` is per consumer. A message in workqueue retention can be redelivered up to `max_deliver` times to one consumer.
- **Slow consumers in NATS Core get cut off.** The server doesn't buffer indefinitely. Slow consumers see disconnects.
- **Inbox subject reuse in request/reply.** Long-lived inboxes can leak across requests if the client doesn't manage them.
- **Account isolation is real.** Two accounts can't see each other's subjects without explicit exports. New users sometimes wonder why nothing works across teams.

11. Quick reference

Concept	Key fact
Subject	Dot-separated hierarchical name (<code>orders.us.created</code>)
Wildcard <code>*</code>	Matches one token
Wildcard <code>></code>	Matches one or more tokens, last position only
Queue group	Round-robin across subscribers
Request / reply	RPC-style messaging with reply-to subject
Stream	Persistent log on subjects (JetStream)
Consumer	View of a stream (push or pull)
Retention	<code>limits</code> , <code>interest</code> , <code>workqueue</code>
Replication	R=1, R=3, R=5 (Raft, odd numbers)
Acknowledgement	<code>none</code> , <code>all</code> , <code>explicit</code>
KV store	Key-value layer on JetStream
Object store	Blob layer on JetStream
Leaf node	Edge bridge to a main cluster
Account	Multi-tenant boundary
Java client	<code>io.nats:jnats</code>